

# Les Algorithmes Gloutons

## Définition

Les algorithmes gloutons (ou voraces, (en anglais : greedy)) construisent une solution de façon incrémentale, en choisissant à chaque étape la direction qui est la plus prometteuse. Ce choix localement optimal n'a aucune raison de conduire à une solution globalement optimale.

## Les algorithmes gloutons

- sont assez simples  $\Rightarrow$  rapides
- Généralement utilisés pour la résolution des problèmes d'optimisation
- Les solutions proposées sont approximatives (heuristiques)
- Permettent de fournir

-  Solution optimale
-  Solution approximative non optimale
-  Pas de solution alors qu'elle existe

## Schéma Général

```
Fonction vorace (C : ensemble) : ensemble  
//C est l'ensemble de tous les candidats}  
S  $\leftarrow$   $\emptyset$  //ensemble solution//  
Tant que non solution (S) et C  $\neq \emptyset$  faire  
    x  $\leftarrow$  l'élément de C qui maximise select(x)  
    C  $\leftarrow$  C \ {x}  
    Si réalisable (S  $\cup$  {x}) alors  
        S  $\leftarrow$  S  $\cup$  {x}  
    Finsi  
Fin tant que  
Si solution(S) alors retourner (S)  
Sinon retourner  $\emptyset$  (pas de solution)  
Finsi  
Fin
```

## Exemple : Minimum de pièces de monnaie

### Problème

Remettre la monnaie en donnant le moins de pièces possibles

Soient

- C : ensemble fini de pièces
- Solution(S) : total des pièces choisies correspondant au montant à rendre
- Ensemble réalisable : total des pièces n'excède pas la somme à rendre
- Fonction de sélection la plus grande pièce qui reste dans C
- Fonction objective : nombre de pièces utilisées dans la solution

La fonction remise monnaie peut être décrite comme suit :

**Fonction Remise-Monnaie(Montant) :tableau**

```
//C est l'ensemble de tous les candidats exemple C :{1,5,10,20,50,100} //
S ← 0
Tant que (Montant > 0) et C ≠ ∅ Faire
    x ← max {y / y ∈ C}
    C ← C \ {x}
    NP ← Montant div x // nombre de pièces de type x //
    S[x] ← NP
    Montant ← Montant - NP * x
Fin tant que
Si Montant = 0 alors retourner (S)
Sinon retourner(pas de solution)
Finsi
Fin
```

**Exemple 1**

C : {1,5,10,12,20,50,100}

M=16

Résultat de l'algorithme

12 + 1 + 1 + 1 + 1

→ 1 pièce de 12 et 4 pièces de 1

→ 5 pièces solution non optimale

Autre solution (non fournie par l'algorithme)

10 + 5 + 1

→ 3 pièces solution optimale

**Exemple 2**

C : {5,10,12,20,50,100}

M=15

Résultat de l'algorithme

12 + ?

→ pas de solution

une solution existe

10 + 5

→ 2 pièces solution optimale

### Autre version : Problème Minimum de pièces de Monnaie

// M :montant à atteindre// //C est l'ensemble de tous les candidats//

#### Algorithme Remise monnaie

```
A ← 0
Np ← 0
S[x] ← 0
Tant que ( A < M) Faire
    Choisir la pièce x qui a la plus grande valeur /  $A+x \leq M$ 
    A = A+x
    Np = Np+1
    S[x] ← S[x]+1
Fin tant que
Si A=M alors solution (S,Np)
Sinon pas de solution
Finsi
```

#### Exemple 1

C :{1,2,5}

M=10

Résultat de l'algorithme

5 + 5



2 pièces

solution optimale

#### Exemple 2

C :{4,6}

M=8

Résultat de l'algorithme



6 + ?

blocage

pas de solution

Une solution existe

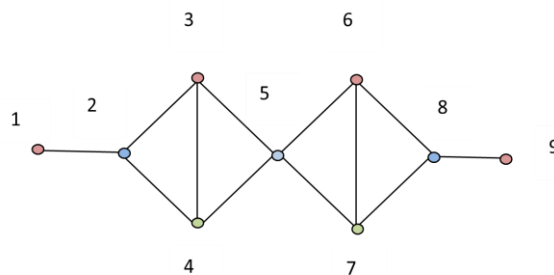
4 + 4



2 pièces

solution optimale

#### Exemple : coloration 1



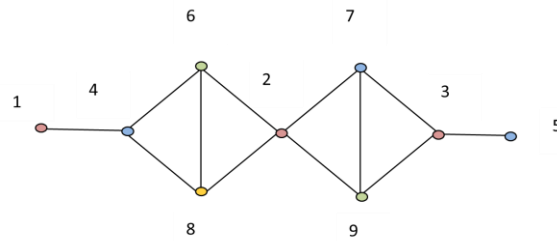
| Ordre de traitement des sommets | 1   | 2    | 3   | 4     | 5    | 6   | 7     | 8    | 9   |
|---------------------------------|-----|------|-----|-------|------|-----|-------|------|-----|
| Couleur                         | red | blue | red | green | blue | red | green | blue | red |

Résultat de l'algorithme:

3 couleurs ont été utilisées pour colorer ce graphe

**Solution optimale**

### Exemple : coloration 2



| Ordre de traitement des sommets | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---------------------------------|---|---|---|---|---|---|---|---|---|
| Couleur                         |   |   |   |   |   |   |   |   |   |

Résultat de l'algorithme:

4 couleurs ont été utilisées pour colorer ce graphe

**Solution réalisable non optimale**

### Exemple : Recherche de plus courts chemins

#### Problème

Recherche d'une arborescence des plus courts chemins issue d'un sommets s du réseau  $R(X,E,d)$

Résolution Algorithme glouton  $\Rightarrow$  **Algorithme de Dijkstra**

#### Algorithme de Dijkstra

*/\*Cet algorithme est traité à titre d'exemples d'algorithmes glouton il n'est pas demandé de le connaître juste pour avoir une idée et voir les résultats fournis \*/*

**But de cet algorithme :** Déterminer une arborescence des plus courts chemins issue d'un sommet s relativement au réseau  $R(X,U,d)$ .

Soient  $\pi(x)$  : la valeur du plus court chemin de s à x.  
 $A(x)$  : l'arc qui a fourni la meilleure valeur  $\pi(x)$   
 sracine est une variable logique qui vaut Vrai ssi s est racine

Cet algorithme ne donne un résultat correct que si  $d \geq 0$ . Dans le cas où les distances sont quelconques ( $>0, <0, =0$ ) l'arborescence obtenue n'est pas toujours l'arborescence des plus courts chemins.

**Algorithme de Dijkstra (Données :  $X, U, I, T, d, s$  ; Résultats :  $\pi, A, \text{sracine}$ )**

/\*Initialisation \*/

$S := \{s\}$  ;  $\pi(s) := 0$  ;  $A(s) := -1$  ;  $x_{\text{pivot}} := s$  ;  $\pi(x) := +\infty$  pour tout  $x \in X, x \neq s$  ;

/\*S est un ensemble qui va comporter les sommets pivots dont la valeur de  $\pi$  a été calculée, à chaque itération un sommet est ajouté à cet ensemble\*/

/\* La valeur d'un sommet choisi pivot ne doit pas être modifiée au cours des itérations futures. Les valeurs des  $\pi(x)$  des sommets non sélectionnés pivots peuvent être améliorées. Dans notre contexte l'amélioration est une réduction de la valeur.\*/

/\* traitement itératif \*/

Tant que  $S \neq X$  et  $\pi(x_{\text{pivot}}) < \infty$  Faire

    Pour tout  $u \in U / I(u) = x_{\text{pivot}}$  et  $T(u) \notin S$  Faire

$x := T(u)$

        Si  $\pi(x) > \pi(x_{\text{pivot}}) + d(u)$  alors  $\pi(x) = \pi(x_{\text{pivot}}) + d(u)$

$A(x) := u$

    Fin si

Fin pour

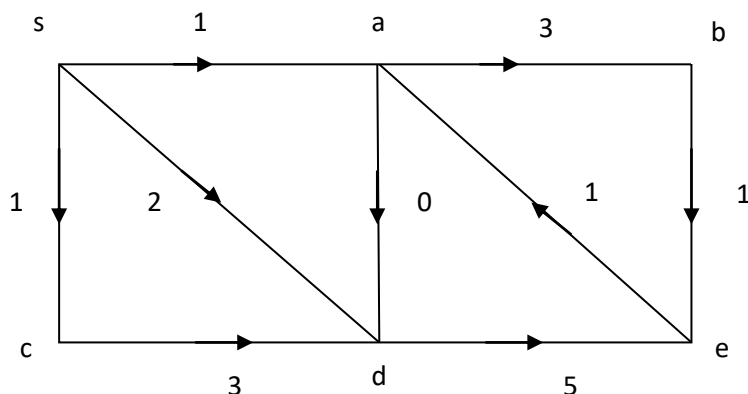
    choisir  $x \notin S / \pi(x) := \min_{y \notin S} \pi(y)$  ;   /\* en cas d'exaequo le choix est arbitraire\*/

$x_{\text{pivot}} := x$  ; si  $\pi(x) < \infty$  alors  $S := S \cup \{x_{\text{pivot}}\}$  fsi

Fin tant que

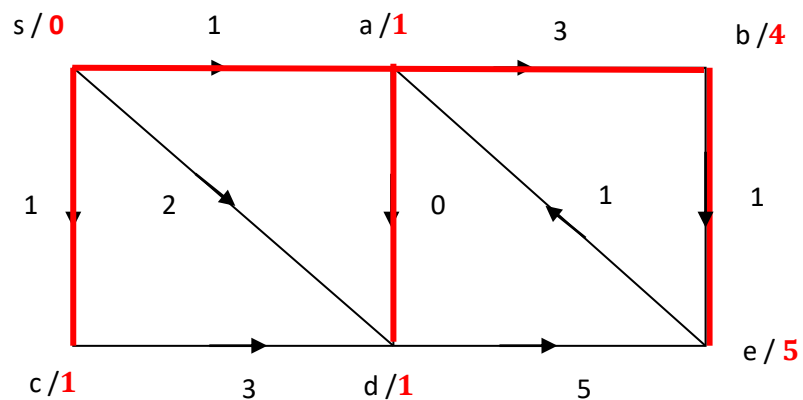
Si  $S = X$  alors  $\text{sracine} := \text{vrai}$  sinon  $\text{sracine} := \text{faux}$  Finsi

Fin

**Exemple 1**

## Application de l' Algorithme Dijkstra

| Itération      | S | a        | b        | c        | d        | e        | xp | A(xp) | S             |
|----------------|---|----------|----------|----------|----------|----------|----|-------|---------------|
| initialisation | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | s  | -1    | {s}           |
| Itération1     | 0 | 1        | $\infty$ | 1        | 2        | $\infty$ | a  | (sa)  | {s,a}         |
| Itération 2    | 0 | 1        | 4        | 1        | 1        | $\infty$ | c  | (sc)  | {s,a,c}       |
| Itération 3    | 0 | 1        | 4        | 1        | 1        | $\infty$ | d  | (ad)  | {s,a,c,d}     |
| Itération 4    | 0 | 1        | 4        | 1        | 1        | 6        | b  | (ab)  | {s,a,b,c,d}   |
| Itération 5    | 0 | 1        | 4        | 1        | 1        | 5        | e  | (be)  | {s,a,b,c,d,e} |

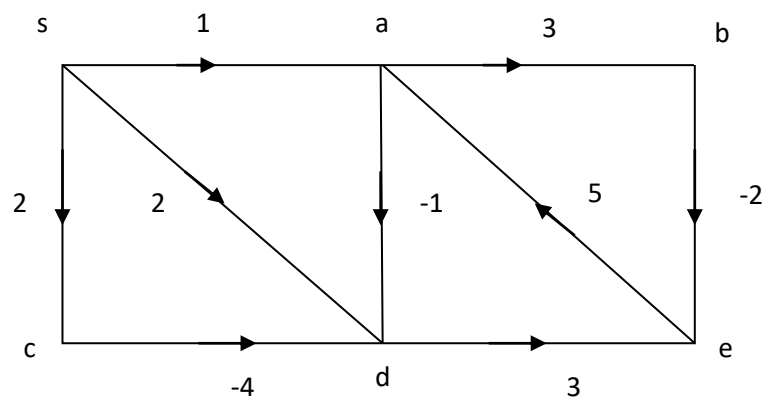


Fin du traitement itératif avec  $S=X$  alors  $s_{racine} := \text{vrai}$ . On a déterminé, ainsi, une arborescence des plus courts chemins issue de s. Les arcs en rouge forment cette arborescence.

Les valeurs des distances sont  $\geq 0$  ➡ la solution obtenue est optimale

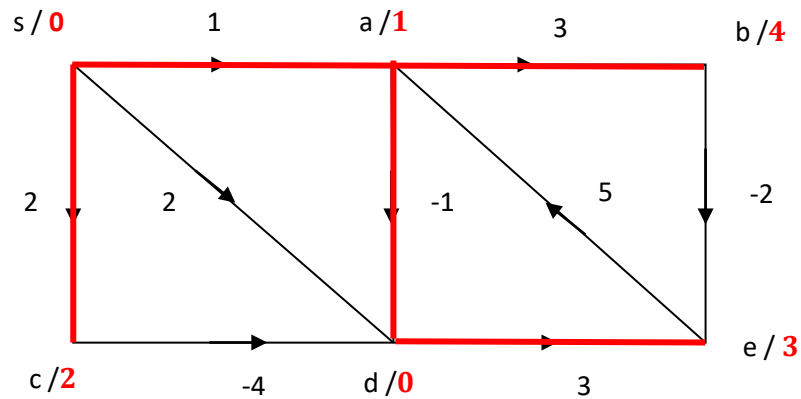
### Exemple 2

On considère le réseau suivant et on cherche à déterminer une arborescence des plus courts chemins issue de s:



## Application de l'Algorithme de Dijkstra

| Itération      | S | A        | b        | c        | d        | e        | xp | A(xp) | S             |
|----------------|---|----------|----------|----------|----------|----------|----|-------|---------------|
| initialisation | 0 | $\infty$ | $\infty$ | $\infty$ | $\infty$ | $\infty$ | s  | -1    | {s}           |
| Itération1     | 0 | 1        | $\infty$ | 2        | 2        | $\infty$ | a  | (sa)  | {s,a}         |
| Itération 2    | 0 | 1        | 4        | 2        | 0        | $\infty$ | d  | (ad)  | {s,a,d}       |
| Itération 3    | 0 | 1        | 4        | 2        | 0        | 3        | c  | (sc)  | {s,a,c,d}     |
| Itération 4    | 0 | 1        | 4        | 2        | 0        | 3        | e  | (de)  | {s,a,c,d,e}   |
| Itération 5    | 0 | 1        | 4        | 2        | 0        | 3        | b  | (ab)  | {s,a,b,c,d,e} |



Les valeurs des distances sont quelconques ( $\geq 0$  et  $\leq 0$ ) la solution obtenue peut être optimale ou non. Dans cet exemple l'arborescence obtenue n'est en fait qu'une solution réalisable non optimale puisqu'il existe une solution optimale donnée par l'arborescence des plus courts chemins issue de s décrite ci-dessous :

